



MINI PROJECT • COMPLETE WALKTHROUGH REPORT

# Quiz Rush

A real-time, multiplayer trivia game where every question is generated live from today's technology news — not a static question bank.

REACT • NODE.JS • SOCKET.IO • REDIS • SUPABASE • GOOGLE GEMINI

# Table of Contents

---

|    |                                                 |
|----|-------------------------------------------------|
| 01 | Executive Summary                               |
| 02 | Problem Statement & Motivation                  |
| 03 | Objectives                                      |
| 04 | Technology Stack                                |
| 05 | System Architecture                             |
| 06 | The Core Gameplay Loop                          |
| 07 | Development Journey — Project Stages            |
| 08 | Key Features                                    |
| 09 | Security & Fairness — Anti-Cheat Design         |
| 10 | The Question Pipeline — From Headline to Trivia |
| 11 | Database Schema Overview                        |
| 12 | Testing & Quality Assurance                     |
| 13 | Challenges Faced & Solutions                    |
| 14 | Deployment                                      |
| 15 | Future Scope                                    |
| 16 | Conclusion                                      |

## 01 Executive Summary

---

**QuizRush** is a real-time, multiplayer trivia application built around a simple premise: trivia should be about *right now*, not a recycled question bank. Players create or join a room, pick topics they care about, and answer a live round of questions that were written minutes earlier from real technology news headlines by a large language model.

The system pairs a React single-page application with a Node.js/Socket.io real-time backend, Redis for fast ephemeral game state, Supabase (PostgreSQL) for durable accounts and history, and the Google Gemini API for question generation from live RSS feeds.

This report documents the project end-to-end: the problem it addresses, its architecture, the chronological stages of its development — including the performance, cost, and security issues that were discovered and fixed along the way — its feature set, its testing strategy, and its deployment path.

## 02 Problem Statement & Motivation

---

Most casual trivia games — digital or otherwise — draw from a fixed, pre-written question bank. That has two consequences: questions get stale and repeat quickly, and the game has no real connection to what is actually happening in the world on the day it's played.

Separately, tech-savvy groups of friends and colleagues already follow the news cycle closely — new model releases, funding rounds, security disclosures, hardware launches. There was no lightweight, social way to turn "did you see today's tech news?" into a genuine competitive game.

QuizRush addresses both problems at once: it treats the live news cycle as its question bank, regenerating trivia on demand from whatever topics a room's players choose, so the content is always current and never the same round twice.

### CORE INSIGHT

Reusing an LLM's ability to *read* news and *write* a fair quiz question from it turns an otherwise static genre (trivia) into a live one — provided the generation pipeline is fast and cheap enough to run per-room, and the scoring is trustworthy enough for real competition.

## 03 Objectives

---

- **Live content:** generate every question from real, recent news articles rather than a static bank.
- **True real-time multiplayer:** every player in a room sees the same question at the same instant, with a synchronized countdown.
- **Fair, tamper-resistant scoring:** a player's score must depend only on facts the server itself can verify — never on a value the client self-reports.
- **Operate within free-tier limits:** the LLM and database usage patterns must stay well inside a free-tier quota and free-tier storage, since this is a self-funded mini project, not a funded product.
- **Distinct, considered visual design:** avoid the generic "AI-template" look common to quickly-built apps, in favour of a UI with real typographic and interaction character.
- **Format variety:** avoid limiting the game to plain multiple-choice; support several distinct ways of answering a question.

## 04 Technology Stack

---

### 4.1 Client

React 19 Vite Tailwind CSS v4 Framer Motion React Router 7 Socket.io Client  
Supabase JS

### 4.2 Server

Node.js Express Socket.io Redis (ioredis) Supabase (PostgreSQL)  
Google Gemini API rss-parser Zod

### 4.3 Tooling & Quality

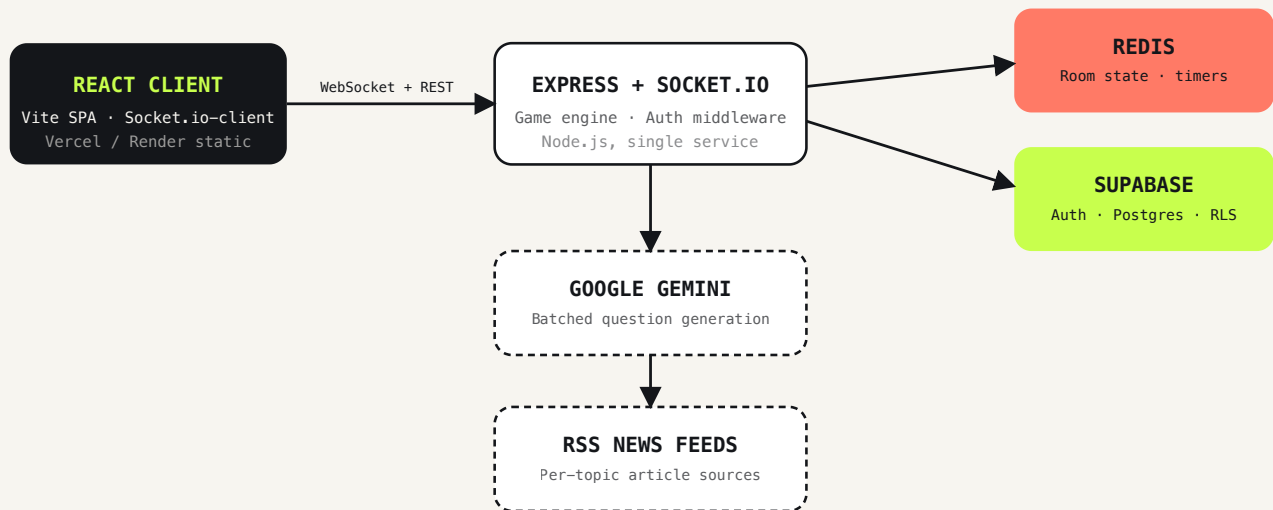
Jest ESLint Docker docker-compose Render (hosting)

#### WHY THIS STACK

Socket.io gives bidirectional, room-scoped real-time events with automatic reconnection; Redis holds fast-changing per-room game state so Postgres is never in the hot path of a live round; Supabase supplies auth, durable storage, and row-level security without hand-rolling an auth server.

## 05 System Architecture

The system separates *durable* state from *ephemeral, fast-moving* state. Anything that must survive a server restart or feed a leaderboard lives in Supabase; anything that changes every few hundred milliseconds during a live round — timers, in-flight answers, room rosters — lives in Redis.



### 5.1 Why a single Express + Socket.io service

In production, the Express server also serves the client's built static assets directly, so the whole application deploys as one service — a deliberate simplification appropriate to a mini project's scale, avoiding the operational overhead of coordinating two separately-deployed services.

## 06 The Core Gameplay Loop

---

1. **Sign in** — Google OAuth (via Supabase Auth), or join anonymously with just a room code for a one-off session.
2. **Onboarding** (first-time accounts) — choose a nickname, an avatar tile colour, and the news topics you follow.
3. **Create or join a room** — the host picks topics, difficulty, question count, and seconds-per-question; joiners enter a 5–6 character room code.
4. **Waiting room** — shows the room code, a shareable link, and the live roster of joined players.
5. **Live round** — for each question: the server broadcasts it to everyone at once, a synchronized countdown runs, players answer, and a reveal shows the correct answer, an explanation, and the running leaderboard. A standings screen appears at the halfway point.
6. **Results** — final placement, a podium, accuracy/speed/streak stats, and the questions you missed.
7. **Profile** — lifetime score and games played, plus a breakdown of accuracy by answer format for the game just played.



## 07 Development Journey — Project Stages

The project did not proceed as a single linear build. It moved through distinct stages, each triggered by a concrete problem surfacing in the previous one — the pattern of a real iterative build rather than a one-shot implementation.

1

### STABILIZATION

#### Codebase Audit & Bug Fixes

Started from an existing frontend/backend skeleton. A structural audit surfaced and fixed several concrete bugs: an incomplete `room:join` response that omitted most of the room state, a broken `StrictMode` import, missing per-player correct-answer tallies, and a lint setup that was configured but not actually installed. Dead, unused components and a broken favicon reference were also cleared out.

2

### PERFORMANCE & COST

#### Question-Generation Pipeline Overhaul

The original pipeline fetched news and called the LLM once per question, sequentially — slow, and prone to filling a free-tier database quickly since every game snapshotted its full question set into Postgres. Rewrote it to fetch all RSS feeds concurrently, batch many articles into a single Gemini call (multiplying effective quota several times over), cache the resulting pool in Redis keyed by topic combination, and generate on demand per room instead of on a fixed schedule. Game-ending cleanup now nulls the heavy question snapshot in Postgres while keeping the lightweight score history.

3

### CONFIGURATION

#### LLM Quota & Model Selection

Diagnosed why the Gemini free tier felt so restrictive: quota is tied to the API key's underlying Google Cloud project (raised by linking Cloud Billing), not to a personal subscription such as Google One. Also caught and corrected a model choice — `gemini-1.5-flash` — that had since been fully retired, replacing it with a current, larger-quota flash-tier model.

4

## CORRECTNESS &amp; SECURITY

**Full Codebase Review**

A dedicated review pass, done independent of any specific bug report, surfaced a genuine scoring exploit: the server trusted a client-reported "time remaining" value when computing speed-based points, so a modified client could always claim maximum score. Fixed by having the server record its own broadcast timestamp per question and clamp any claimed timing against it — a client can end up scored lower than it claims, never higher. The same pass fixed a question-pool sizing bug, two places where a blunt `socket.off(event)` call was silently removing every listener for that event rather than just one, and two unhandled promise rejections on Supabase writes.

5

## DESIGN EXPLORATION

**Moving Past the "Generic AI Template" Look**

The original interface, while functional, read as a generic template rather than a considered design. Three fully distinct visual directions were prototyped — an editorial "broadcast newsroom" look, a competitive "esports scoreboard" look, and a quiet, restrained late-night look — each carrying the same underlying idea (live news turned into trivia) through a different personality, for review before committing to one.

6

## DIRECTION &amp; ITERATION

**Converging on a Final Design**

One direction was refined into a final interactive prototype: a warm paper-and-ink palette with a lime/coral accent pair, Space Grotesk and JetBrains Mono typography, and support for five distinct answer formats (not just multiple choice) plus wager and steal scoring mechanics. In parallel, real implementation work began directly against the production codebase — a new Tailwind design-token theme, the beginnings of multi-format question generation, and draft wager/steal scoring logic — ahead of the prototype being fully finalized.

## 7

## RECONCILIATION &amp; COMPLETION

**Implementing the Final Design End-to-End**

Bringing the finished prototype into the real codebase required first auditing exactly what had already been built in parallel, since it was substantially further along than expected but only half-wired: the server generated a question *format* field that the socket layer never actually forwarded to clients, and a wager/steal choice the client collected was accepted by the API but silently dropped before reaching the server. Closing those gaps, then building the screens the prototype called for but the app still lacked — nickname/topic onboarding, a mid-game standings screen, and a player profile/statistics page — completed the redesign. A persisted light/dark theme system replaced a broken per-page toggle; a per-game cap and real symmetric risk were added to the wager/steal mechanics; eleven fully orphaned legacy components were removed.

## 8

## VERIFICATION

**Testing, Linting, and Build Verification**

The full change set was verified with a clean ESLint pass across both client and server, a green Jest suite (fixing two pre-existing test suites whose mocks predated the multi-format question schema, and adding new tests for the wager/steal fixes), and a successful production build.

## 08 Key Features

### 8.1 Five Answer Formats

| FORMAT         | HOW YOU ANSWER                          | SCORING                                                                                              |
|----------------|-----------------------------------------|------------------------------------------------------------------------------------------------------|
| <b>Single</b>  | Pick one of several options             | Full credit for the correct option                                                                   |
| <b>Multi</b>   | Pick every option that applies          | Full credit for an exact set match, partial credit for a partially correct, no-wrong-picks selection |
| <b>Rank</b>    | Put a short list into the correct order | Full credit for an exact order, partial credit otherwise                                             |
| <b>Swipe</b>   | True / False on a single claim          | Full or no credit                                                                                    |
| <b>Type-in</b> | Type a short free-text answer           | Full credit on a match, partial credit on a close/contained match                                    |

### 8.2 Risk Mechanics

#### Wager

Double your points for a correct answer — or lose the equivalent amount for a wrong one. A genuine bet, not a free upgrade.

#### Steal

Once per game, name an opponent before answering. If you answer correctly and they answer wrong, a fixed amount transfers from their score to yours.

### 8.3 Other Features

- Persisted light/dark theme, detected from system preference on first visit.
- Guest play via room code — no account required.
- Host-configurable topics, difficulty, question count, and seconds per question.
- Mid-game standings screen at the halfway point of a round.
- Post-game profile analytics: accuracy by answer format and a "weak spot" callout.
- Instant rematch — recreate a room with the same settings in one tap.

## 09 Security & Fairness — Anti-Cheat Design

---

A real-time scoring game is only as fair as its least-trusted input. Speed-based scoring in particular creates an obvious incentive to lie about how quickly an answer was submitted.

### 9.1 The Vulnerability

The original answer-submission event included a client-computed `timeRemainingMs` value, and the server used it directly to compute points. Because this value originates entirely on the client, a modified client (or a direct socket call bypassing the UI) could submit any answer along with a fabricated "answered instantly" claim and receive maximum points regardless of actual response time.

### 9.2 The Fix

The server now records its own timestamp the moment a question is broadcast to a room. When an answer arrives, it computes the true remaining time from that server-side clock and takes the *lower* of the client's claim and the server's own calculation. A client can therefore only ever be scored lower than what it honestly earned by being slow over the network — never higher by lying.

#### SAME PRINCIPLE, APPLIED TWICE MORE

The one-steal-per-game limit and the symmetric wager risk are both enforced the same way — as server-side state the client cannot see or influence directly, rather than trusting a client-sent flag.

## 10 The Question Pipeline — From Headline to Trivia

---

1. Per-topic RSS feeds are fetched concurrently the moment a room is created, so generation happens in the background while players are still joining.
2. Articles are grouped into small batches, and each batch is turned into several finished questions with a *single* Gemini call — the key lever for making a strict free-tier daily request quota go much further.
3. The model is asked to vary the answer format per question (single / multi / rank / swipe / type-in) and to return a strict JSON schema, which is then validated, re-shuffled (so the correct option isn't predictably positioned), and shaped into the app's internal question format.
4. Finished questions are cached in Redis, keyed by a hash of the room's topic selection, so the next room requesting the same topics reuses the pool instead of paying for regeneration.
5. At game start, a game-sized set is randomly selected from the pool and locked to that room, so every player in the room receives the identical set of questions.

# 11 Database Schema Overview

| TABLE         | PURPOSE                                    | NOTABLE COLUMNS                                                                       |
|---------------|--------------------------------------------|---------------------------------------------------------------------------------------|
| profiles      | One row per account, extends Supabase auth | username, tile_color, fav_topics, total_score, games_played, onboarded                |
| rooms         | A hosted game room                         | code, host_id, topics, question_count, difficulty, question_time_seconds, status      |
| game_sessions | One row per played round                   | room_id, topics, started_at, ended_at (question snapshot cleared after the game ends) |
| scores        | Per-player result for a session            | session_id, user_id, score, rank, correct_count                                       |
| chat_messages | In-room chat history                       | room_id, user_id, message                                                             |

Row-Level Security is enabled on every table: profiles and rooms are publicly readable but only self-editable; scores and chat are publicly readable and insertable only by authenticated users.

## 12 Testing & Quality Assurance

---

- **Unit tests** cover the anti-cheat timing logic (fabricated-claim clamping, honest-answer scoring, the no-timestamp fallback), the wager/steal scoring fixes, and the batched question-generation logic.
- **Integration tests** exercise the Socket.io room-join flow end-to-end against a real socket server.
- **Static analysis** — a full ESLint configuration (including React-hooks-aware rules) runs clean across both the client and server packages.
- **Build verification** — a production Vite build completes successfully with no errors.



13

# Challenges Faced & Solutions

| CHALLENGE                                                          | SOLUTION                                                                                                                   |
|--------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Free-tier LLM quota too small for one-call-per-question generation | Batch several articles into one Gemini call, multiplying effective daily quota                                             |
| Every game's full question set filling a free-tier database        | Null the heavy JSONB snapshot once a game ends, keep only lightweight score history                                        |
| Client-reported timing made scoring trivially fakeable             | Server-recorded broadcast timestamps, authoritative clamping                                                               |
| Generic, characterless first-pass UI                               | Multiple bespoke design directions prototyped, then implemented as a full token-based design system                        |
| Reconciling design work done in parallel, mid-implementation       | A full audit of the actual current codebase before any further change, rather than assuming a prior known state still held |

## 14 Deployment

---

The server is built as a multi-stage Docker image; in production it also serves the client's built static assets, so the whole application ships as a single deployable service. A `docker-compose` file provides a local Redis instance for development. The application is hosted on **Render**, with Supabase and Redis (Upstash) as managed external services, and configuration supplied entirely through environment variables.

## 15 Future Scope

---

- **Lifetime per-format analytics** — persisting a full answer history (not just per-session) to power long-term accuracy trends on the profile page.
- **Server-authoritative streak multiplier** — currently shown for information only; making it affect scoring would need the same server-side tracking discipline as the timing fix.
- **Spectator mode** for rooms that have already started.
- **Cross-room, friends-based leaderboards.**
- **Additional answer formats** and difficulty-adaptive question selection.

## 16 Conclusion

---

QuizRush set out to prove that a trivia game can be genuinely live — sourced from the news of the day rather than a static bank — without sacrificing the fairness a competitive game needs. Getting there required working through real constraints at every layer: a free-tier LLM quota, free-tier database storage, a scoring channel that had to be made tamper-resistant, and a visual identity that had to be designed rather than defaulted to. The result is a complete, tested, deployable application that answers the original brief: real-time, multiplayer, and built on what's actually in the news right now.